

x402: The HTTP-Native Payment Protocol — A Reference for Protocol Engineers

Document version: 2026-05-18 · Audience: blockchain/protocol engineers integrating x402

1. Origins and Design Philosophy

1.1 The dormant 402 status code

HTTP status code **402 "Payment Required"** was reserved in the original HTTP/1.1 specification (RFC 2068, January 1997) as a placeholder for a future micropayment layer that never materialized. Successive revisions — RFC 2616, RFC 7231 §6.5.2, and the current RFC 9110 §15.5.3 — have retained the same brief, open-ended definition: *"The 402 (Payment Required) status code is reserved for future use."* MDN summarises the situation bluntly: "No standard use convention exists and different systems use it in different contexts."

(<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/402>) (MDN Web Docs)

For almost three decades, 402 was effectively unusable. The web's native transport had a slot for payment semantics, but the underlying payment rails — credit cards with \$0.30 minima, batched ACH, custodial APIs gated by accounts and OAuth — could not be embedded into a single request-response cycle. Past attempts to revive 402 fell into two camps: the W3C Web Monetization specification (<https://webmonetization.org/specification/>), driven by the Interledger Foundation, which streams Interledger Protocol packets to a `<link rel="monetization">` payment pointer via a browser extension; and Lightning Labs' **L402** (originally LSAT, <https://github.com/lightninglabs/L402>), which combines a Bitcoin Lightning invoice with a macaroon capability token returned in `WWW-Authenticate: L402 ...`. Neither achieved broad adoption outside their respective ecosystems. (MEXC + 3)

1.2 The x402 announcement

x402 was published by the Coinbase Developer Platform on **6 May 2025**, in a launch blog by Erik Reppel, Nemil Dalal, and Dan Kim (<https://www.coinbase.com/developer-platform/discover/launches/x402>). The launch framed x402 as "an open standard that leverages the original HTTP '402 Payment Required' status code to embed stablecoin payments directly into web interactions," explicitly citing the lineage of Balaji Srinivasan's 21.co work on Bitcoin micropayments and noting that "with modern L2s like Base, onchain fees have dropped to 1 cent, so many of the applications they prototyped are becoming possible." (Coinbase) (Coinbase)

The stated design rationale is narrow and technical:

1. **Machine-to-machine payments.** Account creation, OAuth, API keys, and minimum-charge fees are friction that autonomous software clients cannot easily traverse. A protocol whose

only authentication artifact is a signed payment authorization removes the need for an identity-bearing relationship between client and server. (Simplescraper) (Goplausible)

- 2. **Stablecoin-native settlement.** Settling in USD-pegged stablecoins (canonically USDC) avoids exchange-rate volatility and provides a familiar unit of account for pricing.
- 3. **Micropayment economics.** Layer-2 rollups (Base) and high-throughput L1s (Solana, Algorand) have driven on-chain settlement cost into the fractions-of-a-cent range, finally making sub-cent transactions economically defensible. (Eco)
- 4. **HTTP-native composition.** All x402 traffic rides on standard HTTP semantics; existing CDN, caching, proxy, and middleware infrastructure works unchanged. (Blockedden)

1.3 Governance: the x402 Foundation

x402 was incubated inside Coinbase but moved toward neutral stewardship in September 2025. On **23 September 2025**, Coinbase and Cloudflare announced the intent to launch the **x402 Foundation** to "create a new standard to simplify the process of sending and receiving payments on the Internet" (<https://www.cloudflare.com/press/press-releases/2025/cloudflare-and-coinbase-will-launch-x402-foundation/>). Matthew Prince, Cloudflare CEO, characterised the partnership in the release as: *"Coinbase deserves immense credit for starting the work on the x402 protocol and we're excited to partner with them on our shared vision for a neutral foundation."* (Ln + 3)

The Foundation was formally constituted under the Linux Foundation on 2 April 2026. Per the Linux Foundation press release, the x402 Foundation was "initially developed by Coinbase, Cloudflare, and Stripe," with founding membership comprising **Adyen, Amazon Web Services, American Express, Base, Circle, Cloudflare, Coinbase, Fiserv Merchant Solutions, Google, KakaoPay, Mastercard, Merit Systems, Microsoft, Polygon Labs, PPRO, Shopify, Sierra, Solana Foundation, Stripe, thirdweb, and Visa**. The canonical repository was moved from (github.com/coinbase/x402) to (github.com/x402-foundation/x402); the Coinbase repo now describes itself as a "development fork." (DEVOPSdigest + 3)

1.4 Comparison with alternative web-payment schemes

Property	x402	L402 (Lightning Labs)	Web Monetization (W3C/Interledger)
Status-code anchor	HTTP 402 + (X-PAYMENT) / (PAYMENT-RESPONSE) headers	HTTP 402 + (WWW-Authenticate: L402)	(<link rel="monetization"> (no 402)
Settlement asset	USDC / USDT / EURC / arbitrary ERC-20, ASA, SPL	Bitcoin (sats) over Lightning	ILP packets (currency-agnostic)

Property	x402	L402 (Lightning Labs)	Web Monetization (W3C/Interledger)
Settlement layer	Base, Ethereum, Polygon, Arbitrum, Optimism, Avalanche, Solana, Algorand	Lightning Network	Interledger STREAM
Auth artifact	Signed <code>transferWithAuthorization</code> / atomic txn group	Macaroon + preimage	Wallet-side payment session
Latency to access	1-3s on Base; ~400ms on Solana; ~3s on Algorand finality	ms (off-chain Lightning)	streaming (continuous)
Trust model	Resource server trusts facilitator for liveness; client signs scope-bound transfer	Client trusts gateway-proxy (Aperture)	Browser trusts wallet provider
Production deployments	Coinbase CDP facilitator, Cloudflare pay-per-crawl, AP2 (Google), Vercel	Lightning Loop, Lightning Pool, Aperture	Coil (defunct as consumer product)

Each design makes a different bet about the underlying money rail. L402 binds to Bitcoin/Lightning; Web Monetization binds to Interledger; x402 binds to EVM/Solana/AVM stablecoin transfers. L402 settles faster (Lightning is millisecond off-chain) but lives only inside Bitcoin; x402's audited transferable-stablecoin settlement aligns with how most contemporary fintech infrastructure already prices, accounts for, and reports value. Stated plainly: x402 wins by virtue of incumbent fintech compatibility (stablecoins, EVM tooling) and major-platform distribution (Coinbase, Cloudflare, Stripe, Google AP2), not by any particular cryptographic novelty. (BingX + 2)

2. Core Protocol Specification

The canonical specification lives at `specs/x402-specification.md` in the foundation repository (<https://github.com/coinbase/x402/blob/main/specs/x402-specification.md>). Two profiles ride underneath: `specs/schemes/exact/scheme_exact_evm.md`, `specs/schemes/exact/scheme_exact_svm.md`, and `specs/schemes/exact/scheme_exact_algo.md`.

2.1 The HTTP flow

The end-to-end exchange is a four-message protocol that can be telescoped to three when the

client already knows the price:

1. **Unpaid request.** Client makes an ordinary HTTP request to a protected resource.
2. **Payment-required response.** The resource server replies with `402 Payment Required` carrying a `PAYMENT-REQUIRED` body (V2) or `accepts` body (V1) describing one or more `PaymentRequirements`. `Cloudflare` `Goplausible`
3. **Paid request.** Client selects one `PaymentRequirements` entry, constructs a `PaymentPayload` whose `scheme` and `network` match, signs it locally, base64-encodes the JSON, and resends the original request with a `PAYMENT-SIGNATURE` header (V2) or `X-PAYMENT` header (V1). `Goplausible`
4. **Resource response.** Resource server verifies the payload (locally or via `POST /verify` to a facilitator), settles the payment (locally or via `POST /settle` to a facilitator), and returns the resource with `200 OK` and a `PAYMENT-RESPONSE` (V2) / `X-PAYMENT-RESPONSE` (V1) header containing a base64-encoded `SettlementResponse`. `Cloudflare` `Goplausible`

2.2 The `PaymentRequirements` object

```
json
{
  "scheme": "exact",
  "network": "eip155:84532",
  "amount": "10000",
  "asset": "0x036CbD53842c5426634e7929541eC2318f3dCF7e",
  "payTo": "0x209693Bc6afc0C5328bA36FaF03C514EF312287C",
  "maxTimeoutSeconds": 60,
  "extra": { "name": "USDC", "version": "2" }
}
```

Field semantics:

- `scheme` — payment scheme identifier. Currently standardized: `exact` (pay an exact specified amount). In scoping: `upto` (usage-metered) and `deferred` (paid after delivery, used by Cloudflare's pay-per-crawl). `Coinbase`
- `network` — V2 uses **CAIP-2** identifiers (`eip155:8453` = Base; `eip155:84532` = Base Sepolia; `solana:5eykt4UsFv8P8NJdTREpY1vzqKqZKvdp` = Solana mainnet; `algorand:wGHE2Pwvdv7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=` = Algorand mainnet). V1 name strings (`base-sepolia`, `solana-devnet`, `algorand-testnet`) remain accepted. `github`
- `amount` (V2) / `maxAmountRequired` (V1) — atomic-unit integer encoded as a string (`"10000"` of USDC = \$0.01).

- `asset` — contract address (EVM), mint address (Solana), or 64-bit ASA ID encoded as string (Algorand: `"31566704"` is mainnet USDC). [github](#) [Circle](#)
- `payTo` — recipient address. For the `exact` scheme on EVM and SVM this is locked into the signature; the facilitator cannot redirect funds. [github](#)
- `maxTimeoutSeconds` — the validity window the server is willing to allow; clients set `validBefore` no later than `now + maxTimeoutSeconds`.
- `extra` — scheme/network-specific augmentation. For EVM/EIP-3009 it carries `name`, `version` (USDC's EIP-712 domain `version`), and optionally `assetTransferMethod` (`eip3009 | permit2 | erc7710`). For Algorand it carries an optional `feePayer` Algorand address. [github](#)
- `outputSchema`, `description`, `contentType` — optional, declarative metadata used by paywall UIs and the Bazaar discovery layer.

The 402 response body (V2) carries an array of these requirements alongside a `resource` block describing the protected endpoint:

```
json
{
  "x402Version": 2,
  "error": "PAYMENT-SIGNATURE header is required",
  "resource": {
    "url": "https://api.example.com/premium-data",
    "description": "Access to premium market data",
    "contentType": "application/json"
  },
  "accepts": [ /* PaymentRequirements[] */ ],
  "extensions": {}
}
```

2.3 The `PaymentPayload` object

```
json
```

```

{
  "x402Version": 2,
  "scheme": "exact",
  "network": "eip155:84532",
  "accepted": { /* echo of the chosen PaymentRequirements */ },
  "payload": {
    "signature": "0x2d6a7588...1c",
    "authorization": {
      "from": "0x857b06519E91e3A54538791bDbb0E22373e36b66",
      "to": "0x209693Bc6afc0C5328bA36FaF03C514EF312287C",
      "value": "10000",
      "validAfter": "1740672089",
      "validBefore": "1740672154",
      "nonce": "0xf3746613c2d920b5fdabc0856f2aeb2d4f88ee6037b8cc5d04a71a4462f13480"
    }
  },
  "extensions": {}
}

```

The `payload` content is **scheme-and-network specific**: on EVM with `assetTransferMethod=eip3009` it is the structure above; on EVM with `permit2` it carries a `permit2Authorization` with `permitted`, `from`, `spender`, `nonce`, `deadline`, and a `witness { to, validAfter, extra }` block; on Solana it is a base64-encoded partially-signed transaction with `TransferChecked` plus `SetComputeUnitLimit` / `SetComputeUnitPrice` instructions; on Algorand it is an atomic transaction group plus a `paymentIndex` pointer. ([github + 2](#))

2.4 The facilitator interface

The facilitator exposes three HTTP endpoints. Their contract is normative (RFC 2119 MUST language is used inside the spec) but their transport is otherwise unconstrained.

```

POST /verify
  { "paymentPayload": ..., "paymentRequirements": ... }
→ { "isValid": bool, "invalidReason"?: string, "payer"?: address }

POST /settle
  { "paymentPayload": ..., "paymentRequirements": ... }
→ { "success": bool, "errorReason"?: string,
    "payer"?: address, "transaction"?: txid,
    "network": caip2 }

GET /supported
→ { "kinds": [ { "scheme": "exact", "network": "eip155:8453", ... } ] }

```

Standard error codes include `invalid_scheme`, `invalid_network`, `invalid_payload`, `invalid_payment_requirements`, `invalid_x402_version`, `insufficient_funds`, `invalid_transaction_state`, and `unexpected_verify_error`. [GitHub](#)

2.5 Header conventions

Header	Direction	Encoding	Purpose
<code>PAYMENT-REQUIRED</code> (V2) / body only (V1)	server → client	base64(JSON)	402 challenge
<code>PAYMENT-SIGNATURE</code> (V2) / <code>X-PAYMENT</code> (V1)	client → server	base64(JSON)	Signed <code>PaymentPayload</code>
<code>PAYMENT-RESPONSE</code> (V2) / <code>X-PAYMENT-RESPONSE</code> (V1)	server → client	base64(JSON)	Settlement receipt

Servers MUST NOT change request semantics on the retry — the retried request is byte-identical to the unpaid one with the addition of the `PAYMENT-SIGNATURE` header.

2.6 Why three parties

The protocol's three-party structure — **client**, **resource server**, **facilitator** — is deliberate:

- The **client** holds the signing key and produces a payload that binds the transfer to a specific `to` address and a specific `value`.
- The **resource server** holds only the recipient address and the route configuration; it never holds the client's funds and never needs to talk to the chain. [DeepWiki](#) [QuickNode](#)

- The **facilitator** holds RPC connectivity, gas, and optionally a fee-payer key. It can drop a transaction, censor it, or charge for the service — but it cannot redirect the payment to itself, because the signed payload binds the recipient. (Algorand)

A resource server may also implement verification and settlement in-process; the facilitator is an optimization, not a requirement.

3. EIP-712 and EIP-3009 Mechanics — the EVM (exact) Scheme

3.1 EIP-3009 transferWithAuthorization

The default EVM transfer method is **EIP-3009** (transferWithAuthorization), defined at <https://eips.ethereum.org/EIPS/eip-3009>. It introduces three EIP-712 typed structures: (TransferWithAuthorization), (ReceiveWithAuthorization), and (CancelAuthorization). The type hash is: (Coinbase)

```
solidity

bytes32 public constant TRANSFER_WITH_AUTHORIZATION_TYPEHASH =
    keccak256(
        "TransferWithAuthorization(address from,address to,uint256 value,"
        "uint256 validAfter,uint256 validBefore,bytes32 nonce)"
    );
// = 0x7c7c6cdb67a18743f49ec6fa9b35f50d52ed05cbcd4cc592e13b44501c1a2267
```

(reference: <https://github.com/CoinbaseStablecoin/eip-3009/blob/master/contracts/lib/EIP3009.sol>)

The function signature on the token contract is:

```
solidity

function transferWithAuthorization(
    address from, address to, uint256 value,
    uint256 validAfter, uint256 validBefore, bytes32 nonce,
    uint8 v, bytes32 r, bytes32 s
) external;
```

3.2 Why EIP-3009, not EIP-2612

EIP-3009 differs from the more widely deployed EIP-2612 ((permit)) in two consequential ways:

The cost of this choice is that EIP-3009 must be implemented natively in the token contract. Circle implements EIP-3009 in USDC v2 across all supported chains; Tether does not, and has stated no plans to. For non-EIP-3009 tokens the spec defines a **Permit2** fallback (see §3.5).

DEV Community Coinbase

3.3 The EIP-712 domain separator

The domain is constructed from `name`, `version`, `chainId`, `verifyingContract`:

```
solidity
DOMAIN_SEPARATOR = keccak256(abi.encode(
    keccak256("EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)",
    keccak256(bytes(name)),           // e.g. "USD Coin"
    keccak256(bytes(version)),        // e.g. "2"
    chainId,
    verifyingContract
));
```

Some bridged tokens (notably Polygon's USDC.e, contract `0x2791bca1f2de4661ed88a30c99a7a9449aa84174`) implement a *different* domain structure with a `salt` rather than `chainId`; this is a known footgun, and the EVM `exact` scheme does not support bridged USDC on Polygon — only Circle-native USDC at `0x3c499c542cEF5E3811e1192ce70d8cC03d5c3359`.

3.4 Canonical USDC contract addresses

Network	CAIP-2	USDC contract
Ethereum mainnet	eip155:1	<code>0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48</code> <code>MEXC</code> <code>Bitget</code>

Network	CAIP-2	USDC contract
Base mainnet	<code>eip155:8453</code>	<code>0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913</code> Go Packages Circle
Base Sepolia	<code>eip155:84532</code>	<code>0x036CbD53842c5426634e7929541eC2318f3dCF7e</code> Go Packages
Polygon (native)	<code>eip155:137</code>	<code>0x3c499c542cEF5E3811e1192ce70d8cC03d5c3359</code>
Arbitrum One (native)	<code>eip155:42161</code>	<code>0xaf88d065e77c8cC2239327C5EDb3A432268e5831</code> Circle
Optimism (native)	<code>eip155:10</code>	<code>0x0b2C639c533813f4Aa9D7837CAf62653d097Ff85</code>
Avalanche C- Chain	<code>eip155:43114</code>	<code>0xB97EF9Ef8734C71904D8002F8b6Bc66Dd9c48a6E</code> Circle

(Cross-referenced against Circle's authoritative list at <https://developers.circle.com/stablecoins/usdc-contract-addresses>.)

3.5 Permit2 and ERC-7710 fallbacks

For tokens that do not implement EIP-3009, the EVM `exact` scheme defines two additional `assetTransferMethod`s:

- **`permit2`** — uses Uniswap's canonical Permit2 contract via the auxiliary `x402ExactPermit2Proxy` deployed at the deterministic CREATE2 address `0x402085c248EaA27D92E8b30b2C58ed07f9E20001`. The user signs a `PermitWitnessTransferFrom` whose `witness` contains the `(to, validAfter, extra)` triple. The proxy enforces that the actual transfer destination equals `witness.to`. A one-time Permit2 allowance (`approve(Permit2)`) is required; this can be paid by the user, sponsored by the facilitator (the `erc20ApprovalGasSponsoring` extension), or replaced with an EIP-2612 permit (`eip2612GasSponsoring` extension).
- **`erc7710`** — uses ERC-7710 delegation managers (e.g., the MetaMask Delegation Framework) to redeem an opaque `permissionContext` against a smart account. Verification is performed by simulating `redeemDelegations`.

3.6 Settlement and reimbursement

The facilitator submits the transaction on-chain and pays gas in the chain's native asset. Reimbursement happens out of band: the facilitator either charges a per-transaction fee

(Coinbase CDP charges \$0.001 per settlement above 1,000/month free), runs at a loss as a customer-acquisition cost (the public `x402.org/facilitator` is free), or recoups the cost via a higher headline price set by the resource server. Critically, **the facilitator cannot deduct from the transferred amount**; the EIP-3009 signature commits `value` and `to` exactly. `Substack`

`Coinbase`

3.7 Security considerations on EVM

- **Nonce uniqueness.** A 256-bit random nonce has negligible collision probability; clients must use a CSPRNG. Replays are blocked because the contract sets `_authorizationStates[from][nonce] = true` on first use.
- **Validity window.** `validAfter` and `validBefore` are absolute seconds-since-epoch. Setting `validBefore` only a few minutes ahead minimizes the window during which a stolen signature could be used.
- **Signature malleability.** Standard `ecrecover` malleability is mitigated by EIP-3009 implementations enforcing low-`s` form (`s <= secp256k1n/2`).
- **Front-running.** Because `transferWithAuthorization` allows any caller, an attacker observing the mempool could race the facilitator. This is harmless to the payer (the transfer goes to the same `to`) but can grief the facilitator into paying for a transaction whose effect is captured by another submitter. Mitigation: use `receiveWithAuthorization` where the protocol allows; submit via a private mempool. `GitHub` `GitHub`
- **Bridged-USDC pitfall.** Always verify that the `verifyingContract` returned in `paymentRequirements.asset` matches Circle-native USDC for the chain.

4. The Facilitator in Detail

4.1 Verification responsibilities

`POST /verify` performs all checks short of broadcasting:

1. Validate `x402Version` is supported (currently `1` or `2`).
2. Validate `scheme` and `network` match between `paymentPayload.accepted` and `paymentRequirements`.
3. Decode the EIP-712 / EIP-3009 message and confirm `ecrecover(signature)` yields `authorization.from`.
4. Confirm `to == payTo`, `value >= amount`, `asset == paymentRequirements.asset`.
5. Confirm the validity window covers the current block time (`validAfter <= now < validBefore`).

6. Confirm the nonce is unused on the token contract (`authorizationState(from, nonce) == false`).
7. Confirm the payer's balance is sufficient.
8. Simulate `transferWithAuthorization(...)` against the token contract; reject on revert.

The verify endpoint is *stateless and idempotent* and is intended to be fast (~100ms over a colocated RPC). (DeepWiki)

4.2 Settlement responsibilities

`POST /settle` submits the transaction. Coinbase's CDP facilitator additionally handles gas estimation, fee-bumping on inclusion delay, nonce management for the broadcaster wallet, retry on transient RPC errors, confirmation polling (~2s typical on Base), and return of a `SettlementResponse` containing `txHash`, `payer`, and `network`. For Algorand (instant finality), the simulate-then-submit pattern collapses to a single `v2/transactions` POST. (DeepWiki + 2)

4.3 Trust model

The facilitator is trusted for **liveness** — it can refuse to verify or settle, returning errors that block the client's access. It is **not trusted for funds**: the EIP-3009 signature binds the transfer to `payTo` and `value`. Even a fully malicious facilitator cannot withdraw to a third address. The strongest attack a facilitator can mount is censorship; the mitigation is the protocol's permissionless design, which allows the resource server (or client) to switch facilitators at any time. (Algorand)

4.4 Hosted vs. self-hosted

Two production options exist as of May 2026:

- **Coinbase CDP facilitator** at `https://api.cdp.coinbase.com/platform/v2/x402` — requires a CDP API key; offers 1,000 free settlements per month, then \$0.001 per transaction; supports Base, Base Sepolia, Ethereum, Polygon, Solana, plus additional EVM networks. (Substack)
- **x402.org/facilitator** — public testnet facilitator, no auth required, supports only Base Sepolia and Solana Devnet. Intended for prototyping. (Coinbase)

Self-hosting is straightforward in the reference SDKs (see §5.1). A community-maintained list of third-party facilitators (Akita, Ultraviolet, GoPlausible, HTTPayer, PayAI, Crossmint) is tracked in the foundation README.

4.5 Fee models in practice

Three patterns are observable: **free / loss-leader** (x402.org, early CDP); **per-transaction fee** (CDP at \$0.001/txn, two orders of magnitude lower than Stripe's \$0.30 floor); **spread on the transfer**

(some third-party facilitators inject a settlement contract that deducts a fee before forwarding to `payTo`). (Medium)

5. The Official Codebase

The monorepo at `github.com/x402-foundation/x402` (mirrored at `github.com/coinbase/x402`) is laid out as:

```
specs/
  x402-specification.md
  schemes/exact/scheme_exact_evm.md
  schemes/exact/scheme_exact_svm.md
  schemes/exact/scheme_exact_algo.md
  extensions/erc20_gas_sponsoring.md
  extensions/eip2612_gas_sponsoring.md
typescript/
  packages/x402/                ← @x402/core
  packages/x402-evm/            ← @x402/evm (EIP-3009, Permit2, ERC-7710)
  packages/x402-svm/            ← @x402/svm (Solana TransferChecked)
  packages/x402-axios/          ← Axios payment interceptor
  packages/x402-fetch/          ← fetch() wrapper
  packages/x402-express/        ← Express middleware
  packages/x402-hono/           ← Hono middleware (Cloudflare Workers)
  packages/x402-next/           ← Next.js middleware + route wrapper
  packages/x402-paywall/        ← Browser-rendered paywall UI
  packages/x402-extensions/     ← Bazaar discovery, SIWx
python/
  x402/                         ← async core
  x402/http/middleware/{fastapi,flask}.py
  x402/http/clients/{httpx,requests}.py
  x402/mechanisms/{evm,svm,avm}/exact/
go/ http/ mechanisms/evm/ mechanisms/svm/
java/
examples/typescript/...
examples/python/...
```

The Coinbase fork carries 5,700+ stars and ~1,300 forks as of May 2026. (GitHub)

5.1 TypeScript reference SDK

```
typescript
```

```

import express from "express";
import { paymentMiddleware, x402ResourceServer } from "@x402/express";
import { ExactEvmScheme } from "@x402/evm/exact/server";
import { HTTPFacilitatorClient } from "@x402/core/server";

const facilitatorClient = new HTTPFacilitatorClient({
  url: "https://x402.org/facilitator",
});
const server = new x402ResourceServer(facilitatorClient)
  .register("eip155:84532", new ExactEvmScheme());

app.use(paymentMiddleware({
  "GET /api/data": {
    accepts: [{
      scheme: "exact", price: "$0.001",
      network: "eip155:84532", payTo: "0xYourAddress",
    }],
    description: "Pay-per-call data API",
  },
}, server));

```

Client side:

typescript

```

import axios from "axios";
import { wrapAxiosWithPayment, x402Client } from "@x402/axios";
import { registerExactEvmScheme } from "@x402/evm/exact/client";
import { privateKeyToAccount } from "viem/accounts";

const client = new x402Client();
registerExactEvmScheme(client, {
  signer: privateKeyToAccount(process.env.EVM_PRIVATE_KEY as `0x${string}`),
});
const api = wrapAxiosWithPayment(axios.create({ baseURL }), client);
await api.get("/api/data"); // 402 → sign → retry → 200

```

5.2 Python reference SDK

python

```
from x402 import x402Client
from x402.mechanisms.evm.exact import register_exact_evm_client

client = x402Client()
register_exact_evm_client(client, signer=my_signer)
async with client.fetch("https://api.example.com/paid") as r:
    data = await r.json()
```

Package extras: `pip install "x402[fastapi]"`, `pip install "x402[flask]"`, `pip install "x402[evm]"`, `pip install "x402[svm]"`, `pip install "x402[all]"`.

5.3 The Bazaar discovery layer

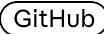
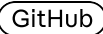
The Bazaar is a service-discovery overlay built on x402's extension mechanism. Resource servers declare input/output schemas in their route configuration; facilitators index those declarations after settlement and expose paginated and semantic-search APIs:

```
GET /v2/x402/discovery/resources?limit=50&offset=0 # paginated catalog
GET /v2/x402/discovery/search?query=weather+api # semantic search
```

A resource only enters the CDP Bazaar index *after* its first successful settlement through the CDP facilitator — discovery is downstream of payment, not registration.

6. Algorand / AVM Implementation via GoPlausible

6.1 Repository and stewardship

The Algorand profile is implemented in `github.com/GoPlausible/x402-avm`. GoPlausible — the team behind the Plausible "proof of anything" protocol on Algorand — built the reference packages, documentation, and example code, and authored Coinbase x402 PR #361, which merged the Algorand exact scheme spec (`specs/schemes/exact/scheme_exact_algo.md`) into the main x402 repository in collaboration with the Algorand Foundation. The project page is <https://x402.goplausible.xyz/>.  

6.2 Why AVM is different

Algorand's transaction model differs from EVM in three ways that drive the design:

1. **No EIP-3009 equivalent.** USDC on Algorand is an **Algorand Standard Asset (ASA)**, ID `31566704` on mainnet, `10458941` on testnet (<https://www.circle.com/multi-chain-usdc/algorand>). ASAs are first-class protocol objects, not smart contracts; there is no

`transferWithAuthorization` to call. The AVM scheme instead uses **atomic transaction groups** signed by the client's Ed25519 key. [Circle + 3](#)

2. **Ed25519, not secp256k1.** Algorand addresses are base32-encoded over `sha512/256(publicKey) || checksum`. There is no `ecrecover`; the public key is derivable from the address itself. Signatures are 64 bytes, applied to msgpack-encoded transactions, not EIP-712 typed data. [github](#)
3. **Native group atomicity and pooled fees.** Up to 16 top-level transactions can be grouped; the group either fully commits or fully reverts. Fees can be **pooled** — one transaction in the group can pay the fees for all others (each transaction's `fee` field can be set to 0 as long as some other transaction in the group overpays by an equivalent amount). [github + 2](#)

6.3 The payment group

The scheme leverages pooled fees for facilitator-paid gas. The client constructs a 2-transaction atomic group: [GitHub](#)

- **Index 0** — `pay` transaction from the *facilitator's fee-payer address* to itself, with `amount = 0` and `fee` set to cover both transactions. Unsigned.
- **Index 1** — `axfer` transaction from the *client* to the *resource server* for `paymentRequirements.amount` of `paymentRequirements.asset`, with `fee = 0`. Signed by the client. [github](#)

The full V2 `PAYMENT-SIGNATURE` payload looks like:

json

```

{
  "x402Version": 2,
  "scheme": "exact",
  "network": "algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=",
  "accepted": {
    "scheme": "exact",
    "network": "algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=",
    "amount": "50000000",
    "payTo": "RESOURCESEVERADDRESS...LTSRPAE",
    "maxTimeoutSeconds": 60,
    "asset": "31566704",
    "extra": { "feePayer": "FACILITATORADDRESS...LQCXBZE" }
  },
  "payload": {
    "paymentIndex": 1,
    "paymentGroup": [
      "gaN0eG6JpGZ1Zc0H0K...", // unsigned facilitator fee-payer txn
      "gqNzaWfEQP3J1DI6..." // client-signed axfer
    ]
  }
}

```

6.4 Validity windows: firstValid / lastValid

There is no `validAfter`/`validBefore` epoch field; Algorand transactions use **round numbers**. `firstValid` (`fv`) and `lastValid` (`lv`) bound a 1,000-round window (~50 minutes at Algorand's ~3-second block time). The client fetches `suggestedParams()` from algod before constructing the group; the facilitator's verification re-fetches the current round and rejects groups whose `lv` has passed.

6.5 Facilitator behaviour on Algorand

The Algorand facilitator performs:

1. **Verify** the structure of the group: ≤16 transactions, correct asset, correct receiver, correct amount, sender match on the fee-payer transaction.
2. **Sign** the fee-payer transaction with its Ed25519 key. ([github](#))
3. **Simulate** the now-fully-signed group against algod's `/v2/transactions/simulate` endpoint.
4. **Submit** to `/v2/transactions` on success. ([github](#))
5. **Wait** for confirmation (typically one round, ~3 seconds; Algorand has instant finality once included). ([github](#)) ([GitHub](#))

A critical safety check, enforced verbatim by the spec, is that the facilitator's fee-payer transaction **MUST** be a self-payment with `amount=0` and **MUST NOT** contain `close`, `rekey`, or other rebinding fields. This bounds the facilitator's exposure: even if the client constructs a malicious payment group, the facilitator's signature only authorizes a self-pay-zero. [github](#)

6.6 Logic signatures and delegated signing

Beyond standard Ed25519, Algorand supports **logic signatures** (`lsig`) — TEAL/PuyaTS programs whose hash is the account address. The AVM spec acknowledges that facilitators offering gasless transactions may offload validation to a logic signature program. This is an optional pattern; the canonical flow uses a normal Ed25519 facilitator key. [github](#) [github](#)

6.7 ARC standards in play

- **ARC-4 ABI** — used by any application-call transactions in extension flows; not required for the core `axfer` path.
- **ARC-26 URI scheme** — `algorand://...` URIs, interoperable with x402 pay-to encoding; not strictly part of x402-avm.
- **ARC-19 / ARC-69** — mutable / template NFT metadata standards used by integrations like AgenticPlace. Not part of the payment protocol itself.
- **No ARC for x402-avm specifically**: the protocol is defined at the x402 layer, not as an Algorand ARC.

6.8 Wallet integration

typescript

```
interface ClientAvmSigner {
  address: string;
  signTransactions(
    txns: Uint8Array[],
    indexesToSign?: number[],
  ): Promise<(Uint8Array | null)[]>;
}
```

This matches the signing surface of `@txnlab/use-wallet`, Pera Wallet's signing API, and other AlgoKit-integrated wallets. The x402-avm v2.6 release removed the direct `algosdk` dependency in favor of `@algorandfoundation/algokit-utils@10.0.0-alpha.39`. [GitHub](#) [GitHub](#)

7.1 Replay protection

- **EVM (EIP-3009)** — 256-bit random nonce in a per-account bitmap on the token contract.
- **EVM (Permit2)** — Permit2 maintains a per-account nonce bitmap.
- **Algorand** — implicit in the round-number window plus unique transaction IDs.
- **Solana** — partially-signed transactions carry a `recent_blockhash` that expires after ~150 slots (~60s).

7.2 MEV and front-running

On EVM, a `transferWithAuthorization` payload visible in the mempool can be submitted by any party — the transfer still goes to the legitimate `to`, but an adversary can grief a facilitator by paying gas to "steal" inclusion. Mitigations: private mempool submission (Flashbots, Base's private bundles), `receiveWithAuthorization` where the recipient is itself the on-chain caller. On Algorand and Solana, MEV-style reordering is less profitable due to single-leader block production.

7.3 Facilitator misbehaviour

- **Censorship.** A facilitator can refuse to verify or settle. The signed payload is portable to any other facilitator that supports the scheme.
- **Withholding.** A facilitator could verify, return success to the resource server, and then never broadcast. The resource server must treat `settle` as the final commit point.
- **Fee inflation.** Mitigation: monitor `/supported`, contract with a published fee schedule.
- **Compliance pressure.** Hosted facilitators implement KYT and OFAC screening; the protocol itself does not. `CryptoSlate`

7.4 Client-side risks

Wallets that present an opaque hash to the user are unsuitable for x402. Agent runtimes that sign autonomously must enforce per-request and aggregate spending limits.

7.5 Privacy

x402 inherits the on-chain visibility of its underlying networks. Every settlement is a public transfer of USDC between two addresses; observers correlating IP addresses, request patterns, and on-chain transfers can reconstruct who paid for what. The pseudonymity is meaningfully weaker than L402's, where the macaroon and Lightning preimage are not on-chain.

8. Adoption, Ecosystem, and Deployments

8.1 Volume and trajectory

CryptoSlate, reporting CDP's own end-of-year figures, states: *"By December, it had processed 75 million transactions worth \$24 million for paid APIs and AI agents."*

(<https://cryptoslate.com/what-is-x402-the-http-402-payments-standard-powering-ai-agents-explained/>). KPMG's independent analysis through February 2026 recorded **161.32 million cumulative transactions, \$43.57 million settled volume, 417,000 buyers, and 83,000 sellers**. Solana reports 35 million x402 transactions by March 2026; Base accounts for the majority (~119 million cumulative as of January 2026). (CryptoSlate + 4)

These headline figures are contested. CoinDesk (11 March 2026, <https://www.coindesk.com/markets/2026/03/11/coinbase-backed-ai-payments-protocol-wants-to-fix-micropayment-but-demand-is-just-not-there-yet>) quotes an Artemis analyst directly: *"The x402 'agent payments' boom is still mostly a mirage. Recent daily snapshots show about 131,000 transactions generating roughly \$28,000 in volume, with the average payment worth around \$0.20."* Artemis estimates approximately half of observed transactions are "gamified" self-trading rather than genuine commerce. The contradiction between headline and organic volume is unresolved at the time of writing; engineers should price-in that the protocol's installed base is real but its sustained transactional demand is materially smaller than coalition-cited totals suggest. (CoinDesk) (CoinDesk)

8.2 Notable integrations as of May 2026

- **Google AP2 (Agent Payments Protocol)** — uses x402 as the on-chain settlement extension for agent-to-agent stablecoin payments. (Allium)
- **Cloudflare** — integrating a deferred-payment x402 scheme into the pay-per-crawl beta and the Agents SDK. (Coinbase)
- **Visa Trusted Agent Protocol (TAP)** — announced October 2025, supports x402 as a settlement rail. (ChainCatcher)
- **Anthropic MCP** — x402 is the canonical payment path for paid tools exposed via MCP servers in Claude Desktop.
- **Amazon Bedrock AgentCore Payments** — natively integrates the x402 discovery layer and wallet infrastructure. (Coinbase)
- **Stripe** — x402 Foundation co-founder per the Linux Foundation launch; integration roadmap not yet public. (Unchained)
- **NEAR Intents** — cross-chain settlement bridge into x402.
- **Vercel** — x402-MCP integration for monetised AI endpoints.
- **Chainlink** — VRF demos paying USDC via x402 on Base Sepolia. (Coinbase)

8.3 Bazaar discovery and x402scan

The **x402 Bazaar** (<https://docs.cdp.coinbase.com/x402/bazaar>) is the CDP-indexed discovery layer; it surfaces paid endpoints indexed off CDP-facilitated settlements. **x402scan** is the public analytics indexer.

8.4 Comparative adoption vs. L402 and Web Monetization

L402 has six years of production history but a much smaller transactional footprint, concentrated in Lightning-native developer tooling. Web Monetization remains the smallest of the three by active integration count; its core consumer product (Coil) is defunct and the protocol survives primarily through the Interledger Foundation's open-source plugins. By cumulative transaction volume in early 2026, x402 has eclipsed both predecessors by at least an order of magnitude, though the gap on *organic* (non-wash) volume is narrower than the headline figures suggest. (Ln)

9. PYTHAI Ecosystem Deployment Context

This section documents the deployment target on the PYTHAI side of the integration. It is intentionally brief; the protocol material above is the canonical part of this document.

9.1 AgenticPlace as the registry layer

AgenticPlace (<https://agenticplace.pythai.net>) is described on its own front page as "the living ERC-8004 agent registry. Index, mint, verify, and explore the autonomous agent economy across every chain." It indexes the global ERC-8004 registry across 48 chains and extends it with Algorand-native agent minting, verification, and oracle attestation (the `agenticORacle` / aORC subsystem). The on-chain registry anchor is the EVM contract

`0x8004a169fb4a3325136eb29fa0ceb6d2e539a432`; Algorand agents are issued as ASAs with ARC-69 metadata (aNFT / dNFT / iNFT / THOT). The chain mapping itself is served by `agenticplace.pythai.net/allchain.html` and the underlying `/api/allchainft` endpoint. The agent registration UI surfaces a wallet picker with two options — Pera and Parsec — and the directive-protocol pricing tiers are denominated in ALGO (`base 0.001 ALGO`, `phi 0.001618 ALGO`, `phi-sq 0.002618 ALGO`, `phi-cu 0.004236 ALGO`). x402 payment gating is exposed as a filter (`/api/agents?x402=...`) on the registry. `pythai + 3`

9.2 mindX as the metered API surface

mindX (<https://mindx.pythai.net>) is the orchestration framework described in the PYTHAI ecosystem index as "Augmentic intelligence orchestration." Its public source lives at `github.com/abaracadabra/mindX`. In the x402 deployment, mindX provides the metered HTTP surface — endpoints whose `GET` and `POST` handlers are wrapped by the `@x402-avm/express` or

`x402-avm[fastapi]` middleware with `paymentRequirements` denominated in USDC ASA `31566704` (or in BANKON, see §9.3) and paid to a mindX-controlled `payTo` address. The mindX runtime emits structured choice logs ("Gödel choice schema and global log") that constitute the metered work product; each paid call corresponds to a logged inference, retrieval, or reasoning step inside mindX. `GitHub + 2`

9.3 BANKON as the on-chain payment unit

BANKON (<https://bankon.pythai.net>) is the Qubic quantum price oracle and the Algorand Standard Asset minted at ASA ID `203977300` on mainnet. BANKON is paired with the mindX inference marketplace as the value-bearing token; `x402` `paymentRequirements.asset` fields can be set to either USDC (`31566704`) for stable settlement or to BANKON (`203977300`) for native ecosystem pricing. The reference is the **BANKON PYTHAI** GitHub organisation (<https://github.com/BANKONPYTHAI>). BANKON is *not* deployed on EVM; there is no Foundry contract corresponding to it. The Solidity-on-Foundry side of the PYTHAI deployment is reserved for the EVM-side ERC-8004 agent registry and any future cross-chain bridge artifacts.

`github` `LabLab`

9.4 DAIO as the on-chain endpoint type

In PYTHAI usage, **DAIO** expands to "Decentralized Autonomous Intelligent Organization" — distinct from generic DAO terminology and from unrelated DAIO usages in the broader blockchain world. The DAIO reference source is `github.com/Professor-Codephreak/DAIO`, with related orgs at `w3DAIO` and `mldaio`. A DAIO is, in protocol terms, a *resource server identity*: an Algorand account (with ASA opt-ins to USDC and BANKON), an `x402` facilitator binding, and a mindX-backed handler. A DAIO appears in the AgenticPlace registry as an ERC-8004-compliant agent record and accepts payments through the standard `x402-avm` flow described in §6. From the protocol's perspective, a DAIO is simply a `payTo` address with discovered routes; from the PYTHAI runtime's perspective, it is the on-chain endpoint of metered or paid agent calls. `github`

9.5 The Parsec wallet

The **Parsec wallet** is a PYTHAI-built sovereign Algorand wallet, built in Tauri + Rust, described in the Professor Codephreak ecosystem index as "PARSEC — sovereign universal wallet ... inspired by bankonVAULT + smartmetamask." It exposes a **Pera-compatible signing interface** over WebSocket/JSON-RPC 2.0 so existing Algorand dApps interoperate without changes. In the `x402` deployment, Parsec implements the `ClientAvmSigner` contract described in §6.8 — `signTransactions(txns, indexesToSign)` returning Ed25519-signed msgpack blobs for the client-controlled transactions in the `x402-avm` payment group. Parsec is distinct from `parsec.finance` (defunct DeFi analytics terminal), `parsec.app` (remote desktop), and the `PARS` Parsec coin. `pythai + 2`

9.6 Tooling: Foundry, AlgoKit, NFDomains

- **Foundry** is the canonical Solidity test framework for the EVM side of the deployment — the ERC-8004 registry artifacts, any future bridge contracts, and any EVM-side x402 receiver contracts are written and tested under Foundry. Mainnet deployment targets are Base (`(eip155:8453)`) for low-cost USDC settlement and Ethereum (`(eip155:1)`) for canonical registry anchoring.
 - **AlgoKit** is the canonical Algorand tooling. The AVM x402 stack uses `@algorandfoundation/algokit-utils@10.0.0-alpha.39` directly. AgentiPlace's `a0RC Minter` and `a0RC Registry` contracts are written in **PuyaTS**, AlgoKit's TypeScript smart-contract language, and deployed to Algorand mainnet using the AlgoKit CLI. `GitHub`
 - **NFDomains** is the Algorand name service used to resolve human-readable names to `payTo` addresses. The protocol-level pattern is straightforward: a DAIO's NFD resolves to the same Algorand address used as `paymentRequirements.payTo`, providing a stable identifier for the x402 endpoint.
-

10. Open Questions and Known Limitations

- **Wash-trading versus genuine commerce.** Independent indexers consistently report that a significant fraction (roughly half per Artemis) of x402 settlement volume in late 2025 / early 2026 was self-generated or experimental. The protocol provides no native mechanism to distinguish. `CoinDesk`
- **No native refunds or chargebacks.** Settlement is final once the on-chain transaction confirms. Merchant-initiated compensating transfers must be implemented at the application layer. `CryptoSlate`
- **Token coverage is USDC-shaped.** EIP-3009 is implemented natively only by USDC, EURC, PYUSD, USDP, and FDUSD. The Permit2 fallback closes much of the gap but requires the user (or a sponsor) to pay a one-time approval, and is materially more complex to integrate.
- **No standardised SIWx today.** The Sign-In-With-X header for wallet-controlled session reuse was still scoping as of May 2026 — sessionless per-request payment is the only production-stable pattern. `X402`
- **Compliance is downstream of the facilitator.** The protocol is neutral; hosted facilitators apply jurisdictional rules.
- **Discovery is settlement-gated.** A route does not appear in the CDP Bazaar until at least one settlement has flowed through the CDP facilitator. Self-hosted facilitators must maintain their own catalogs.

The combination of HTTP-native plumbing, EIP-3009 finality, and major-platform distribution makes x402 the most likely candidate to finally consume the long-dormant 402 status code into

general use. The remaining questions are economic and operational rather than cryptographic: whether real machine-to-machine commerce materialises at the volumes the protocol's coalition is projecting, and whether the facilitator layer remains genuinely permissionless as adoption concentrates beneath a small number of hosted operators.